

ucmism2t Developer Guide

UML Class Model to Text - Developer Documentation

Version: 1.0.0
Last Updated: February 2026

Table of Contents

- 1. [Project Overview](#)
- 2. [Directory Structure](#)
- 3. [Prerequisites Installation](#)
- 4. [Eclipse Setup](#)
- 5. [Importing the Project](#)
- 6. [Understanding Key Files](#)
- 7. [Working with Acceleo Templates \(MTL\)](#)
- 8. [Working with Java Services](#)
- 9. [Running and Testing](#)
- 10. [Building with Maven](#)
- 11. [Using the CLI](#)
- 12. [Troubleshooting](#)
- 13. [Additional Resources](#)

Project Overview

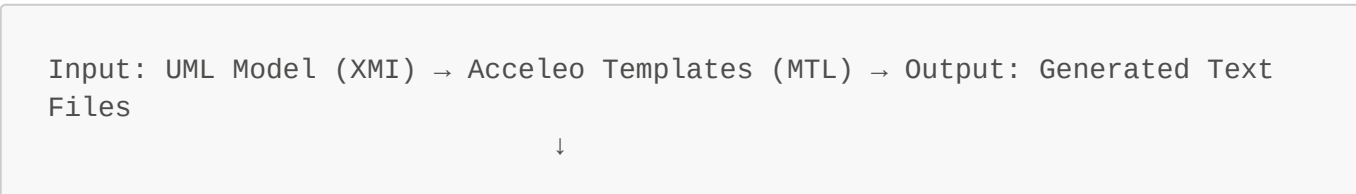
What is ucmism2t?

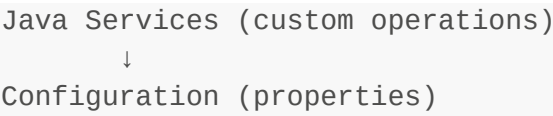
ucmism2t (UML Class Model to Text) is a model-to-text transformation tool that generates text output from UML models using the Acceleo 4.2 template engine. It's built on the Eclipse platform and can be used both as an Eclipse plugin and as a standalone command-line application.

Key Features:

- Reads UML 2.5 models in XMI format
- Uses Acceleo 4.2 templates with AQL (Acceleo Query Language)
- Extensible via custom Java services
- Configurable via external properties files
- Generates any text-based output (code, documentation, schemas, etc.)
- Cross-platform support (Linux, macOS, Windows)

Architecture:





Directory Structure

ucmism2t/	
├─ bundles/	# OSGi bundles (Eclipse plugins)
│ └─ ucmism2t.core/	# Core transformation logic
│ │ └─ META-INF/	
│ │ └─ MANIFEST.MF	# OSGi bundle manifest
│ │ └─ plugin.xml	# Extension point registrations
│ │ └─ src/	
│ │ └─ ucmism2t/	
│ │ │ └─ services/	# Java services for templates
│ │ │ │ └─ ConfigurationService.java	
│ │ │ │ └─ DateTimeService.java	
│ │ │ │ └─ CustomServices.java	
│ │ │ └─ templates/	# Acceleo MTL templates
│ │ │ └─ m2t.mtl	# Main template file
│ │ └─ queries/	# Acceleo query libraries (optional)
│ └─ pom.xml	
├─ ucmism2t.cli/	# Command-line interface
│ │ └─ META-INF/	
│ │ └─ MANIFEST.MF	
│ │ └─ plugin.xml	# Application definition
│ │ └─ src/	
│ │ │ └─ ucmism2t/cli/	
│ │ │ └─ Main.java	# CLI entry point
│ └─ pom.xml	
├─ features/	# Eclipse feature definitions
│ └─ ucmism2t.feature/	
│ │ └─ feature.xml	# Feature configuration
│ └─ pom.xml	
├─ releng/	# Release engineering
│ └─ ucmism2t.target/	# Target platform definition
│ │ └─ ucmism2t.target	# Dependency specifications
│ └─ pom.xml	
├─ ucmism2t.product/	# Product configuration
│ │ └─ ucmism2t.product	# Product definition
│ │ └─ create-single-file.sh	# Packaging script
│ └─ pom.xml	
└─ pom.xml	# Parent Maven POM
└─ README.md	# Project documentation
└─ ECLIPSE_SETUP.md	# Eclipse-specific setup guide

Key Directories Explained

bundles/ucmism2t.core/

- Contains Acceleo templates and Java services
- This is where you develop transformation logic
- Templates in `src/ucmism2t/templates/*.mtl`
- Services in `src/ucmism2t/services/*.java`

bundles/ucmism2t.cli/

- Command-line interface implementation
- Coordinates model loading and template execution
- Usually doesn't need modification unless changing CLI behavior

releng/ucmism2t.target/

- Defines Eclipse and Acceleo versions
- Critical for dependency resolution
- Must be set as active target platform in Eclipse

Prerequisites Installation

1. Java 21 (LTS)

Required Version: Java 21 or later

Used in Development: 21.0.7

Vendor: OpenLogic-OpenJDK

Runtime Path: `/usr/lib/jvm/openlogic-openjdk-21-hotspot-amd64`

Installation:

Linux (Ubuntu/Debian):

```
# Download and install OpenLogic OpenJDK 21
wget https://builds.openlogic.com/downloadJDK/openlogic-
openjdk/21.0.7+7/openlogic-openjdk-21.0.7+7-linux-x64.tar.gz
sudo tar -xzf openlogic-openjdk-21.0.7+7-linux-x64.tar.gz -C /usr/lib/jvm/
sudo update-alternatives --install /usr/bin/java java
/usr/lib/jvm/openlogic-openjdk-21-hotspot-amd64/bin/java 1
sudo update-alternatives --install /usr/bin/javac javac
/usr/lib/jvm/openlogic-openjdk-21-hotspot-amd64/bin/javac 1
```

Verify Installation:

```
java -version
# Expected output:
```

```
# openjdk version "21.0.7" 2024-10-15 LTS
# OpenJDK Runtime Environment (build 21.0.7+7-LTS)
# OpenJDK 64-Bit Server VM (build 21.0.7+7-LTS, mixed mode, sharing)
```

Alternative JDK Vendors:

- Eclipse Temurin (AdoptOpenJDK)
- Oracle OpenJDK
- Amazon Corretto
- Azul Zulu

2. Maven 3.9.9+

Minimum Version: 3.9.9

Used in Development: 3.9.11

Installation:

Linux:

```
# Download Maven
wget https://dlcdn.apache.org/maven/maven-3/3.9.11/binaries/apache-maven-3.9.11-bin.tar.gz

# Extract
sudo tar -xzf apache-maven-3.9.11-bin.tar.gz -C /opt/

# Add to PATH (add to ~/.bashrc or ~/.profile)
export PATH=/opt/apache-maven-3.9.11/bin:$PATH
export M2_HOME=/opt/apache-maven-3.9.11
```

Verify Installation:

```
mvn -version
# Expected output:
# Apache Maven 3.9.11
# Maven home: /opt/apache-maven-3.9.11
# Java version: 21.0.7, vendor: OpenLogic, runtime: /usr/lib/jvm/openlogic-openjdk-21-hotspot-amd64
```

3. Eclipse IDE

Required Version: Eclipse 2025-12 or later

Used in Development: 2025-12

Recommended Variant: Eclipse IDE for RCP and RAP Developers

Download:

<https://www.eclipse.org/downloads/packages/>

Select Package:

```
Eclipse IDE for RCP and RAP Developers
Tools for developers working on Eclipse plug-ins,
RCP applications and RAP applications.

Includes:
- Plugin Development Environment (PDE)
- Maven Integration (m2e)
- Tycho Integration
- Target Platform Management
```

Installation:

```
# Extract Eclipse
tar -xzf eclipse-rcp-2025-12-*.tar.gz -C ~/

# Create desktop launcher (optional)
cat > ~/.local/share/applications/eclipse.desktop << EOF
[Desktop Entry]
Type=Application
Name=Eclipse IDE
Exec=$HOME/eclipse/eclipse
Icon=$HOME/eclipse/icon.xpm
Terminal=false
Categories=Development;IDE;
EOF
```

First Launch Configuration:

1. Launch Eclipse
2. Select workspace: ~/eclipse-workspace
3. Window → Preferences → Java → Installed JREs
 - Add → Standard VM
 - JRE home: /usr/lib/jvm/openlogic-openjdk-21-hotspot-amd64
 - JRE name: OpenJDK 21
 - Finish
 - ☒ Check it as default
 - Apply and Close

4. Acceleo 4.2

Required Version: Acceleo 4.0 or later

Used in Development: 4.2 (R202602100910)

Repository: <https://download.eclipse.org/acceleo/updates/releases/4.2/R202602100910/>

Installation in Eclipse:

Step 1: Open Install Dialog

Help → Install New Software...

Step 2: Add Update Site

Click "Add..." button

Name: Acceleo 4.2

Location:

<https://download.eclipse.org/acceleo/updates/releases/4.2/R202602100910/>

Click "Add"

Step 3: Select All Acceleo Features

In the available software list, you'll see:

- ☒ Acceleo
 - ☒ Acceleo (Required)
 - ☒ Acceleo Developer Resources
 - ☒ Acceleo Editor
 - ☒ Acceleo Query Language (AQL)
 - ☒ Acceleo Query Developer Resources
 - ☒ Acceleo Source

Select ALL items by checking the top-level "Acceleo" box

Step 4: Install

Click "Next"

Review licenses

Accept license agreements

Click "Finish"

Step 5: Trust Unsigned Content

⚠ Warning dialog appears:
"Warning: You are installing software that contains unsigned content..."

☒ Check "Trust unsigned content"
Click "Trust Selected"

Step 6: Restart Eclipse

Click "Restart Now" when prompted

Verify Installation:

After restart:
Window → Preferences → Acceleo

You should see Acceleo preferences with options for:

- Code generation
- Editor preferences
- Template syntax

5. Maven Integration for Eclipse (m2e)

Required Version: 2.0 or later

Used in Development: 2.10

Repository: <https://download.eclipse.org/releases/2025-12/>

Note: This is usually included in "Eclipse IDE for RCP and RAP Developers" but verify it's present.

Verify Installation:

Window → Preferences → Maven

If you see Maven preferences, it's already installed. ✓

If Missing, Install:

1. Help → Install New Software...
2. Work with: <https://download.eclipse.org/releases/2025-12/>
3. Search for: "Maven"
4. Select:
 - ☒ m2e - Maven Integration for Eclipse
 - ☒ m2e - PDE Integration (for Tycho)
5. Click "Next" → "Finish"

6. Trust unsigned content if prompted
7. Restart Eclipse

6. Uninstall Old Tycho Connector

Why: Conflicting Tycho/m2e connectors cause lifecycle mapping warnings.

Check if Present:

```
Help → About Eclipse IDE
Click "Installation Details"
Installed Software tab
Search for: "tycho" or "sonatype"
```

If you find:

- `org.sonatype.tycho.m2e` (old connector)
- `Tycho Configurator` (old connector)

Uninstall it:

1. Select the old connector in the list
2. Click "Uninstall..."
3. Follow wizard
4. Restart Eclipse when prompted

Keep:

- `org.eclipse.m2e.pde.connector` (new connector) ✓
- Any m2e components ✓

Eclipse Setup

Complete Setup Checklist

After installing all prerequisites, configure Eclipse:

1. Set Java 21 as Default

```
Window → Preferences → Java → Installed JREs
☒ OpenJDK 21
Apply and Close
```

2. Configure Maven


```
Window → Preferences → Maven
☒ Download repository index updates on startup
☒ Do not automatically update dependencies from remote repositories
Apply and Close
```

3. Verify Acceleo

```
Window → Preferences → Acceleo
Should show Acceleo configuration options ✓
```

4. Configure Text Encoding (Optional but Recommended)

```
Window → Preferences → General → Workspace
Text file encoding: UTF-8
Apply and Close
```

Importing the Project

Step 1: Import as Maven Project

```
File → Import...
Maven → Existing Maven Projects
Click "Next"
```

Browse to Project:

```
Root Directory: /path/to/ucmism2t
```

Select All Projects:

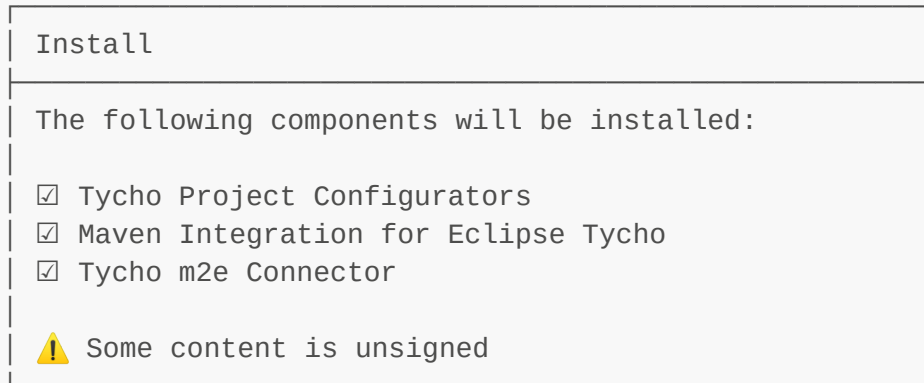
```
Projects (select all 6):
☒ ucmism2t (parent pom)
☒ ucmism2t.core
☒ ucmism2t.cli
☒ ucmism2t.feature
☒ ucmism2t.target
☒ ucmism2t.product
```

```
Click "Finish"
```

Step 2: Automatic Tycho Components Installation

What Happens: Eclipse will automatically download and install Tycho components for Eclipse plugin development.

Dialog Appears:



Actions:

1. ☒ Check "Trust unsigned content"
2. Click "Finish"
3. Wait for installation (progress shown in status bar)
4. Click "Restart Now" when prompted

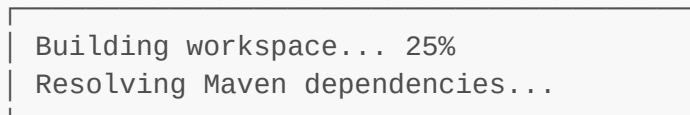
Duration: 2-5 minutes (downloads and installs components)

Step 3: Initial Workspace Build

After restart, Eclipse automatically builds the workspace.

What You'll See:

Bottom-right corner:



Duration: 5-10 minutes on first build

Expected Status:

- Many errors initially (normal - target platform not set yet)
- Maven downloads dependencies
- Projects compile

Step 4: Update Project Configuration

Purpose: Synchronize Eclipse's view of the projects with Maven/Tycho configuration.

Steps:

1. Select all projects in Project Explorer
(Click first project, Shift+Click last project)
2. Right-click → Maven → Update Project...
3. In dialog:
Projects:
☒ ucmism2t
☒ ucmism2t.core
☒ ucmism2t.cli
☒ ucmism2t.feature
☒ ucmism2t.target
☒ ucmism2t.product

Options:
☒ Update project configuration from pom.xml
☒ Refresh workspace resources from local filesystem
☐ Clean projects (we'll do this separately)
☒ Force Update of Snapshots/Releases ← Important!
4. Click "OK"

Duration: 2-5 minutes

What This Does:

- Re-reads all pom.xml files
- Updates Eclipse classpath from Maven dependencies
- Resolves any configuration mismatches
- Downloads latest versions if using force update

Step 5: Clean Projects

Purpose: Remove all compiled artifacts and rebuild from scratch to ensure consistency.

Steps:

- Project → Clean...
- Clean:
- Clean all projects
- Options:
- ☒ Start a build immediately
 - ☒ Build the entire workspace

Click "Clean"

Duration: 3-7 minutes

What This Does:

- Deletes all `.class` files
- Deletes all temporary build artifacts
- Rebuilds everything from source
- Ensures no stale compiled code remains

Monitor Progress:

Bottom-right: "Building workspace... X%"
Problems view: Errors should reduce as build progresses

Step 6: Set Target Platform

Critical Step: This resolves all Eclipse platform dependencies.

Open Target Definition:

In Project Explorer:
ucmism2t/
└─ releng/
 └─ ucmism2t.target/
 └─ ucmism2t.target

Double-click: ucmism2t.target

Target Definition Editor Opens:

ucmism2t.target
Definition Content Environment
Locations:
▼ Eclipse 2025-12 https://download.eclipse.org/releases/2025-12/ Status: [Resolving...]
▼ Acceleo 4.2 https://download.eclipse.org/.../4.2/R202602100910/

A screenshot of a status bar with a light gray background. It contains a rectangular box with a thin black border. Inside the box, the text "Status: [Resolving...]" is displayed in a monospaced font.

Wait for Resolution:**Monitor Progress:**

Bottom-right corner:

A screenshot of a progress bar. It has a light gray background. At the top, the text "Resolving target definition..." is shown. Below it is a progress bar with 10 segments. The first 7 segments are solid black, and the remaining 3 are light gray with a dotted pattern. To the right of the bar, the text "67%" is displayed.

Duration: 5-15 minutes (first time, downloads ~500 bundles)

When Complete:

Locations:

- ▼ Eclipse 2025-12
Status: Resolved ✓ (462 plug-ins available)
- ▼ Acceleo 4.2
Status: Resolved ✓ (41 plug-ins available)

Set as Active:

Look for the link/button in the top-right corner of the editor:

A screenshot of a button with a light gray background and a thin black border. The button contains the text "[Set as Active Target Platform] ← Click" in a monospaced font.

Alternative Method (if link not visible):

Window → Preferences
→ Plug-in Development → Target Platform
→ ☒ ucmism2t.target ← Check this checkbox
→ Apply and Close

Verify Active:

In target editor, you should see at top:
✓ This target definition is currently active

Workspace Rebuild:

After setting the target platform, Eclipse rebuilds:

```
Building workspace... (2-5 minutes)
```

Most errors should disappear after this step!

Step 7: Final Clean

After target platform is set:

```
Project → Clean...  
Clean all projects  
Start a build immediately  
OK
```

Expected Final Status:

```
Problems View:  
- 0 errors ✓  
- 0 warnings ✓  
- 2 infos (lifecycle mapping - ignorable)
```

Understanding Key Files

Acceleo Template: m2t.mtl

Location: `ucmism2t.core/src/ucmism2t/templates/m2t.mtl`

Purpose: Main template file for model-to-text transformation

Structure:

```
[comment encoding = UTF-8 /]  
[module m2t('http://www.eclipse.org/uml2/5.0.0/UML')]  
  
[comment @main /]  
[template public generateModel(aModel : uml::Model)]  
  [file ('output.txt', overwrite)]  
  Model: [aModel.name/]  
  Packages: [aModel.nestedPackage->size()/]  
  [/file]  
[/template]
```

Key Components:

1. Module Declaration:

```
[module m2t('http://www.eclipse.org/uml2/5.0.0/UML')]
```

- Module name must match file name
- URI imports UML metamodel

2. Main Annotation:

```
[comment @main /]
```

- Marks entry point template
- Called automatically by CLI

3. Template Declaration:

```
[template public generateModel(aModel : uml::Model)]
```

- **public**: Visibility
- **generateModel**: Template name
- **aModel**: Parameter name
- **uml::Model**: Parameter type (from UML metamodel)

4. File Generation:

```
[file ('output.txt', overwrite)]  
  Content here  
[/file]
```

- Generates file in output directory
- **overwrite**: Replace if exists

5. AQL Expressions:

```
[aModel.name/]           ← Get property  
[aModel.nestedPackage/]  ← Navigate collection  
[aModel.eAllContents(Class)/] ← Query all classes
```

Java Services

Location: `ucmism2t.core/src/ucmism2t/services/`

ConfigurationService.java

Purpose: Access configuration properties in templates

Key Methods:

```
public String getProperty(Object context, String key)
public String getProperty(Object context, String key, String defaultValue)
public boolean hasProperty(Object context, String key)
```

Template Usage:

```
[aModel.getProperty('project.name')/]
[aModel.getProperty('author', 'Unknown')/]
[if (aModel.hasProperty('debug'))]...[/if]
```

DateTimeService.java

Purpose: Date/time formatting in templates

Key Methods:

```
public String getCurrentDate(Object context)
public String getCurrentTime(Object context)
public String getCurrentDateTime(Object context)
```

Template Usage:

```
Generated on: [aModel.getCurrentDate()/]
Time: [aModel.getCurrentTime()/]
```

CustomServices.java

Purpose: Domain-specific utility methods

Key Methods:

```
public String getQualifiedNamed(NamedElement element, String separator)
public boolean hasStereotype(Element element, String stereotypeName)
```



```
public List<Property> getAttributesByType(Class clazz, String typeName)
```

Template Usage:

```
[cls.getQualifiedName('.')]
[if (cls.hasStereotype('Entity'))]...[/if]
[cls.getAttributesByType('String')->size()]
```

Configuration Properties

Location: Can be anywhere (passed via CLI `-config` argument)

Format: Java Properties

```
# project.properties
project.name=MyProject
author=John Doe
version=1.0.0
package.prefix=com.example.generated
output.encoding=UTF-8
```

Usage in Templates:

```
[comment Project metadata /]
Project: [aModel.getProperty('project.name')]
Author: [aModel.getProperty('author', 'Unknown')]
Version: [aModel.getProperty('version')]

[comment Conditional features /]
[if (aModel.getProperty('generate.tests', 'false') = 'true')]
  [generateTests()]
[/if]
```

Plugin Configuration: plugin.xml

Location: `ucmism2t.core/plugin.xml`

Purpose: Register Java services with Acceleo

Structure:

```
<?xml version="1.0" encoding="UTF-8"?>
<plugin>
  <!-- Register services -->
  <extension point="org.eclipse.acceleo.query.service">
```

```
<service class="ucmism2t.services.ConfigurationService"/>
</extension>

<extension point="org.eclipse.acceleo.query.service">
  <service class="ucmism2t.services.DateTimeService"/>
</extension>

<extension point="org.eclipse.acceleo.query.service">
  <service class="ucmism2t.services.CustomServices"/>
</extension>
</plugin>
```

Important: Any new Java service must be registered here to be available in templates.

OSGi Manifest: MANIFEST.MF

Location: `ucmism2t.core/META-INF/MANIFEST.MF`

Purpose: Define bundle dependencies and exports

Key Sections:

```
Bundle-SymbolicName: ucmism2t.core
Bundle-Version: 1.0.0.qualifier

Require-Bundle:
  org.eclipse.acceleo.aql,
  org.eclipse.acceleo.query,
  org.eclipse.emf.ecore,
  org.eclipse.uml2.uml

Export-Package:
  ucmism2t.services,
  ucmism2t.templates
```

Working with Acceleo Templates (MTL)

Creating a New Template

Step 1: Create .mtl File

```
Right-click: ucmism2t.core/src/ucmism2t/templates/
New → File
File name: mytemplate.mtl
Finish
```

Step 2: Define Module

```
[comment encoding = UTF-8 /]
[module mytemplate('http://www.eclipse.org/uml2/5.0.0/UML')]

[comment @main /]
[template public generate(aModel : uml::Model)]
  [file ('myoutput.txt', overwrite)]
    Content goes here
  [/file]
[/template]
```

Step 3: Save

Eclipse automatically validates syntax.

Template Syntax Basics

Property Access

```
[aModel.name/]
[aClass.isAbstract/]
[aProperty.type.name/]
```

Navigation

[aModel.nestedPackage/]	← Collection of packages
[aClass.ownedAttribute/]	← Collection of attributes
[aOperation.ownedParameter/]	← Collection of parameters

Filtering

```
[comment Get all classes /]
[aModel.eAllContents(Class)/]

[comment Filter by visibility /]
[aClass.ownedOperation->select(op | op.visibility =
VisibilityKind::public)/]
```

Iteration

```
[for (pkg : Package | aModel.nestedPackage)]
  Package: [pkg.name/]
[/for]
```

Conditionals

```
[if (aClass.isAbstract)]
  abstract class
[else]
  concrete class
[/if]
```

Let Bindings

```
[let qualifiedName : String = aClass.qualifiedName]
  QN: [qualifiedName/]
[/let]
```

Complete Example: Java Class Generation

```
[comment encoding = UTF-8 /]
[module javaGen('http://www.eclipse.org/uml2/5.0.0/UML')]

[comment @main /]
[template public generateModel(aModel : uml::Model)]
  [for (pkg : Package | aModel.nestedPackage)]
    [for (cls : Class | pkg.ownedType->filter(Class))]
      [generateClass(cls)/]
    [/for]
  [/for]
[/template]

[template private generateClass(cls : Class)]
  [file (cls.getQualifiedName('/') + '.java', overwrite)]
  package [cls.package.getQualifiedName('.')/];

  [if (cls.isAbstract)]abstract [/if]class [cls.name/] {

    // Attributes
    [for (attr : Property | cls.ownedAttribute)]
      private [attr.type.name/] [attr.name/];
    [/for]

    // Getters
    [for (attr : Property | cls.ownedAttribute)]
      public [attr.type.name/] get[attr.name.toUpperFirst()/]() {
        return this.[attr.name/];
      }
    [/for]
  }
```

```
// Setters
[for (attr : Property | cls.ownedAttribute)]
    public void set[attr.name.toUpperFirst()/]([attr.type.name/]
[attr.name/]) {
    this.[attr.name/] = [attr.name/];
}
[/for]
}
[/file]
[/template]
```

Acceleo Editor Features

Syntax Highlighting:

- Template blocks: `[template]...[/template]` (blue)
- AQL expressions: `[expression/]` (green)
- Comments: `[comment]...[/comment]` (gray)

Content Assist (Ctrl+Space):

```
Type: [aModel.
Press: Ctrl+Space

Shows:
- name : String
- nestedPackage : Package[*]
- ownedType : Type[*]
- eAllContents(EClass) : EObject[*]
- getCurrentDate() : String ← From DateTimeService
```

Error Markers:

- Red X: Syntax error
- Yellow warning: Type mismatch or undefined variable

Quick Fixes:

- Right-click error → Quick Fix
- Suggests corrections

Working with Java Services

Creating a New Service

Step 1: Create Java Class

```
Right-click: ucmism2t.core/src/ucmism2t/services/  
New → Class  
Name: MyService  
Package: ucmism2t.services  
Finish
```

Step 2: Implement Service Methods

```
package ucmism2t.services;  
  
import org.eclipse.uml2.uml.Class;  
import org.eclipse.uml2.uml.NamedElement;  
  
/**  
 * Custom service methods for Acceleo templates.  
 *  
 * All public methods are automatically available in templates  
 * on the type of their first parameter.  
 */  
public class MyService {  
  
    /**  
     * Count attributes in a class.  
     *  
     * Template usage: [aClass.countAttributes()/]  
     *  
     * @param clazz The UML class  
     * @return Number of attributes  
     */  
    public int countAttributes(Class clazz) {  
        return clazz.getOwnedAttributes().size();  
    }  
  
    /**  
     * Convert name to snake_case.  
     *  
     * Template usage: [element.toSnakeCase()/]  
     *  
     * @param element Any named element  
     * @return Name in snake_case format  
     */  
    public String toSnakeCase(NamedElement element) {  
        return element.getName()  
            .replaceAll("[A-Z]", "_$1")  
            .toLowerCase()  
            .replaceFirst("^_", "");  
    }  
}
```

Step 3: Register in plugin.xml

```
<extension point="org.eclipse.acceleo.query.service">
  <service class="ucmism2t.services.MyService"/>
</extension>
```

Step 4: Save and Rebuild

File → Save All
Project → Clean → Clean all projects

Step 5: Use in Template

```
[for (cls : Class | aModel.eAllContents(Class))]  
  [cls.name/] has [cls.countAttributes()/] attributes  
  Table name: [cls.toSnakeCase()/]  
[/for]
```

Service Method Rules

1. First Parameter = Context Type

```
// Called on Class elements  
public String doSomething(Class clazz, String arg) { ... }  
  
// Template: [aClass.doSomething('value')/]
```

2. Return Type

```
// Returns can be:  
public String getValue(...) { ... }           // String  
public int getCount(...) { ... }             // int  
public boolean isValid(...) { ... }           // boolean  
public List<Property> getProps(...) { ... }   // Collections  
public Property getProp(...) { ... }          // UML types
```

3. Multiple Parameters

```
public String format(NamedElement elem, String pattern, boolean upper) {  
  // Template: [element.format('%s_CLASS', true)/]  
}
```

Best Practices

Document Service Methods:

```
/**
 * Clear description of what it does.
 *
 * Template usage: [element.methodName(arg)/]
 *
 * @param element First parameter description
 * @param arg Second parameter description
 * @return What it returns
 */
public String methodName(Type element, String arg) { ... }
```

Handle Null Values:

```
public String getQualifiedName(NamedElement element, String sep) {
    if (element == null) {
        return "";
    }
    String qn = element.getQualifiedName();
    return qn != null ? qn.replace("::", sep) : element.getName();
}
```

Use Descriptive Names:

```
// Good
public boolean hasStereotype(Element element, String name) { ... }

// Bad
public boolean check(Element e, String s) { ... }
```

Running and Testing

Setting Up Run Configuration

Step 1: Create Launch Configuration

```
Run → Run Configurations...
Eclipse Application → New (right-click or click icon)
```

Step 2: Configure Main Tab

Name: ucmism2t Development

Program to Run:

☒ Run an application

Application: ucmism2t.cli.app

Step 3: Configure Arguments Tab

Program arguments:

- input \${workspace_loc}/test-models/sample.xmi
- output \${workspace_loc}/generated
- config \${workspace_loc}/test-configs/test.properties

Working directory:

☒ Default

Note: \${workspace_loc} expands to your workspace path

Step 4: Configure Plug-ins Tab

Launch with: plugins selected below only

Workspace:

- ☒ ucmism2t.core
- ☒ ucmism2t.cli

Target Platform:

- ☒ (all required bundles automatically selected by Eclipse)

Click "Add Required Plug-ins" to ensure all dependencies are included

Step 5: Apply and Run

Click "Apply"

Click "Run"

What Happens When You Run

1. New Eclipse Runtime Launches

A second Eclipse instance starts with your application loaded

2. Console Output Appears

Console (in first Eclipse):

```
Configuration:
  Input model: ../sample.xmi
  Output path: ../generated
  Config files: [test.properties]

Initializing EMF and UML2 framework...
Loading UML model: sample.xmi
Model loaded successfully: MyModel
Executing Acceleo transformation...
✓ Template executed
Generated 1 file(s):
  - output.txt (1245 bytes)
```

3. Output Files Generated

Workspace files refresh automatically:
generated/
└─ output.txt ← Your generated file

Running a Specific MTL File

To use a different template module:

Step 1: Edit Main.java (for permanent change)

```
// In Main.java, find this line:
String moduleQualifiedName = "ucmism2t::templates::m2t";

// Change to your template module:
String moduleQualifiedName = "ucmism2t::templates::mytemplate";
```

Or Step 2: Create New Launch Configuration (for testing)

1. Duplicate existing run configuration
2. Rename: "ucmism2t - MyTemplate"
3. Keep same configuration
4. Edit Main.java temporarily for this module
5. Run the new configuration

Debugging Templates

Set Breakpoints in Java Services:

```
public String toSnakeCase(NamedElement element) {  
    // Click in left margin to set breakpoint here ←  
    String name = element.getName();  
    return name.replaceAll("[A-Z]", "_$1").toLowerCase();  
}
```

Debug Launch:

Debug → Debug Configurations...
Select your run configuration
Click "Debug"

When Breakpoint Hits:

- Variables view shows: element, name, etc.
- Step Over (F6): Execute line
- Step Into (F5): Enter method calls
- Resume (F8): Continue execution

Template Debugging (Limited):

Acceleo doesn't support breakpoints in .mtl files

Instead:

1. Add debug output in template:
[file ('debug.txt', overwrite)]
 Current class: [aClass.name/]
 Attribute count: [aClass.ownedAttribute->size()/]
[/file]
2. Check debug.txt in output directory
3. Use Java service methods and debug those

Common Test Scenarios

Test 1: Simple Model

```
<!-- test-models/simple.xmi -->  
<?xml version="1.0" encoding="UTF-8"?>  
<uml:Model xmi:version="2.0" name="TestModel">  
    <packagedElement xmi:type="uml:Package" name="testpkg">  
        <packagedElement xmi:type="uml:Class" name="TestClass">  
            <ownedAttribute name="id" type="..." />  
        </packagedElement>  
    </packagedElement>  
</uml:Model>
```

```
</packagedElement>
</uml:Model>
```

Test 2: With Configuration

```
# test-configs/test.properties
project.name=Test Project
author=Developer
version=1.0.0
debug=true
```

Test 3: Multiple Outputs

```
[template public generateModel(aModel : Model)]
  [file ('summary.txt', overwrite)]
    Model Summary
  [/file]

  [file ('details.txt', overwrite)]
    Detailed Information
  [/file]
[/template]
```

Building with Maven

Command-Line Build

Basic Build (Linux only):

```
cd ucmism2t
mvn clean verify
```

Platform-Specific Builds:

```
# Windows
mvn clean verify -Dbuild.platform=windows

# macOS Intel
mvn clean verify -Dbuild.platform=macos-x86

# macOS ARM
mvn clean verify -Dbuild.platform=macos-arm
```

```
# All platforms
mvn clean verify -Dbuild.platform=all
```

Build Output Locations

Full Installation:

```
releng/ucmism2t.product/target/products/ucmism2t.product/
├─ linux/gtk/x86_64/
│   ├── ucmism2t                (launcher executable)
│   ├── ucmism2t.ini            (launcher configuration)
│   ├── plugins/                (100+ JAR files)
│   │   ├── org.eclipse.osgi_*.jar
│   │   ├── org.eclipse.acceleo.aql_*.jar
│   │   ├── ucmism2t.core_1.0.0.jar
│   │   └── ...
│   ├── configuration/
│   │   └── config.ini
│   └── features/
```

Archived Product:

```
releng/ucmism2t.product/target/products/
├─ ucmism2t.product-linux.gtk.x86_64.tar.gz
```

Single-File Executable:

```
releng/ucmism2t.product/target/
├─ ucmism2t-standalone (self-extracting executable)
```

Build Phases Explained

1. Validate (validates pom.xml)

```
mvn validate
```

2. Compile (compiles Java sources)

```
mvn compile
```

3. Test (runs unit tests if present)

```
mvn test
```

4. Package (creates JAR bundles)

```
mvn package
```

5. Verify (runs integration tests, creates products)

```
mvn verify ← This is what you typically run
```

6. Install (installs to local Maven repo)

```
mvn install
```

Troubleshooting Maven Builds

Problem: Build fails with "package org.eclipse.uml2.uml does not exist"

Solution: Target platform resolution issue

```
# Clean and rebuild
mvn clean verify

# If still fails, check network connection to Eclipse repositories
curl -I https://download.eclipse.org/releases/2025-12/
```

Problem: Out of memory during build

Solution: Increase Maven memory

```
export MAVEN_OPTS="-Xmx2048m -XX:MaxPermSize=512m"
mvn clean verify
```

Problem: Build succeeds but executable doesn't run

Solution: Check Java version

```
# Executable requires Java 21
java -version
# Should show 21.x.x
```

Using the CLI

Command-Line Syntax

Basic Usage:

```
./ucmism2t -input <model.xmi> -output <directory>
```

With Configuration:

```
./ucmism2t -input model.xmi -output gen/ -config settings.properties
```

Multiple Configurations:

```
./ucmism2t \  
-input model.xmi \  
-output generated/ \  
-config base.properties project.properties
```

Using Full Installation

After Maven build:

```
cd  
releng/ucmism2t.product/target/products/ucmism2t.product/linux/gtk/x86_64/  
  
# Run transformation  
./ucmism2t \  
-input /path/to/model.xmi \  
-output /path/to/output/
```

Using Single-File Executable

Run directly:

```
cd releng/ucmism2t.product/target/  
  
# Make executable (first time only)  
chmod +x ucmism2t-standalone  
  
# Run transformation
```

```
./ucmism2t-standalone \  
-input /path/to/model.xmi \  
-output /path/to/output/
```

Extract for permanent installation:

```
./ucmism2t-standalone --extract ~/ucmism2t-install  
  
# Now use the installation  
~/ucmism2t-install/ucmism2t \  
-input model.xmi \  
-output gen/
```

CLI Arguments Reference

Required Arguments:

Argument	Description	Example
-input <file>	Path to UML model (XMI)	-input model.xmi
-output <dir>	Output directory for generated files	-output generated/

Optional Arguments:

Argument	Description	Example
-config <file> [<file> ...]	Configuration files (properties)	-config base.properties local.properties

Paths:

- Can be absolute or relative
- Relative paths are resolved from current working directory

Examples

Example 1: Basic Code Generation

```
./ucmism2t \  
-input models/system.xmi \  
-output src/generated/
```

Example 2: With Project Configuration

```
./ucmism2t \  
-input models/domain.xmi \  

```



```
-output target/generated-sources/ \  
-config config/project.properties
```

Example 3: Multiple Configurations (Override Pattern)

```
# Base config + environment-specific overrides  
./ucmism2t \  
  -input model.xmi \  
  -output gen/ \  
  -config defaults.properties \  
           production.properties
```

Example 4: CI/CD Integration

```
#!/bin/bash  
# generate.sh - CI/CD script  
  
set -e # Exit on error  
  
MODEL_DIR="models"  
OUTPUT_DIR="src/generated"  
CONFIG="ci-config.properties"  
  
for model in $MODEL_DIR/*.xmi; do  
  echo "Processing: $model"  
  ./ucmism2t \  
    -input "$model" \  
    -output "$OUTPUT_DIR" \  
    -config "$CONFIG"  
done  
  
echo "Generation complete!"
```

Exit Codes

Code	Meaning
0	Success
1	Error (file not found, transformation failed, etc.)

Usage in Scripts:

```
./ucmism2t -input model.xmi -output gen/  
  
if [ $? -eq 0 ]; then  
  echo "✓ Generation successful"
```

```
# Continue with next steps
else
    echo "X Generation failed"
    exit 1
fi
```

Troubleshooting

"Unknown extension point" Errors

Problem:

```
Unknown extension point: 'org.eclipse.acceleo.query.service'
plugin.xml /ucmism2t.core line 37
```

Cause: This is a PDE (Plugin Development Environment) validation issue. The extension point exists in the target platform, but PDE's XML validator can't see it because it only looks in workspace projects.

This is SAFE TO IGNORE - the extension point works perfectly at runtime.

To Remove the Warning (Optional):

Project-Specific Setting (Recommended):

1. Right-click ucmism2t.core project
2. Properties
3. Plug-in Development → Compilers
4. ☒ Enable project specific settings
5. Expand: Extensions and Extension Points
6. Find: Unknown extension point
7. Change from "Error" to "Ignore"
8. Apply and Close

Workspace-Wide Setting:

```
Window → Preferences
→ Plug-in Development → Compilers
→ Extensions and Extension Points
→ Unknown extension point: Error → Ignore
→ Apply and Close
```

Target Platform Not Resolving

Problem: Target definition stuck on "Resolving..." or shows errors.

Solution 1: Check Network

```
# Verify access to Eclipse repositories
curl -I https://download.eclipse.org/releases/2025-12/
curl -I https://download.eclipse.org/acceleo/updates/releases/4.2/
```

Solution 2: Reload Target

1. In target editor, click "Reload Target Platform"
2. Wait for new resolution attempt

Solution 3: Clear P2 Cache

1. Close Eclipse
2. Delete: <workspace>/.metadata/.plugins/org.eclipse.pde.core/.bundle_pool
3. Restart Eclipse
4. Open ucmism2t.target
5. Wait for resolution

Maven Build Fails

Problem:

```
[ERROR] Failed to execute goal org.eclipse.tycho:tycho-compiler-
plugin:5.0.0:compile
```

Solution:

```
# Clean and rebuild
mvn clean verify

# If still fails, ensure Java 21
java -version

# Should show: openjdk version "21.0.7"
```

Template Syntax Errors

Problem: Red X markers in .mtl file

Common Causes:

1. Missing closing tag:

```
[for (cls : Class | model.eAllContents(Class))]  
  [cls.name/]  
  # Missing [/for] here! ← Error
```

Fix: Add closing tag

```
[for (cls : Class | model.eAllContents(Class))]  
  [cls.name/]  
[/for]
```

2. Type mismatch:

```
[let count : String = model.nestedPackage->size()/]  
# Error: size() returns Integer, not String
```

Fix: Correct type

```
[let count : Integer = model.nestedPackage->size()/]
```

3. Undefined variable:

```
[template public generate(aModel : Model)]  
  [aClass.name/] ← Error: aClass not defined  
[/template]
```

Fix: Define in loop

```
[template public generate(aModel : Model)]  
  [for (aClass : Class | aModel.eAllContents(Class))]  
    [aClass.name/]  
  [/for]  
[/template]
```

Java Service Not Available in Template

Problem: Template shows error: "Unknown method: myMethod"

Checklist:

1. Service registered in plugin.xml?

```
<extension point="org.eclipse.acceleo.query.service">
  <service class="ucmism2t.services.MyService"/>
</extension>
```

2. Method is public?

```
public String myMethod(Class clazz) { ... } // ✓ Works
private String myMethod(Class clazz) { ... } // ✗ Not visible
```

3. Workspace rebuilt?

Project → Clean → Clean all projects

4. Correct first parameter type?

```
public String myMethod(Class clazz) { ... }
// Call on Class: [aClass.myMethod()] ✓

public String myMethod(Package pkg) { ... }
// Call on Package: [aPackage.myMethod()] ✓
// Call on Class: [aClass.myMethod()] ✗ Won't work
```

Additional Resources

Official Acceleo 4 Documentation

Main Documentation:

Acceleo 4.2.1 Documentation
<https://github.com/eclipse-acceleo/acceleo/blob/4.2.1/plugins/org.eclipse.acceleo.aql.doc/pages/index.adoc>

Contents:

- Getting Started
- Template Syntax
- AQL Expressions
- File Generation
- Module System
- Best Practices

Acceleo Query Language (AQL) Reference:

```
AQL Documentation
https://github.com/eclipse-
acceleo/acceleo/blob/4.2.1/query/plugins/org.eclipse.acceleo.query.doc/pages/index.adoc
```

Contents:

- AQL Syntax
- Type System
- Collection Operations
- String Operations
- Service Calls
- Lambda Expressions

Useful Links

Eclipse Modeling:

- EMF Documentation: <https://www.eclipse.org/modeling/emf/>
- UML2 Documentation: <https://wiki.eclipse.org/MDT/UML2>
- Eclipse PDE Guide: <https://www.eclipse.org/pde/>

Maven/Tycho:

- Tycho Documentation: <https://tycho.eclipseprojects.io/>
- m2e Documentation: <https://www.eclipse.org/m2e/>

Community:

- Eclipse Forums: <https://www.eclipse.org/forums/>
- Acceleo Discussions: <https://github.com/eclipse-acceleo/acceleo/discussions>
- Stack Overflow: Tag [\[acceleo\]](#) or [\[eclipse-modeling\]](#)

Sample Projects

Acceleo Examples:

```
https://github.com/eclipse-acceleo/acceleo/tree/master/examples
```

UML Model Examples: Check the Eclipse UML2 project for sample .uml files that can be exported to XML.

Appendix: Quick Reference

Common AQL Operations

Collections:

```
[collection->size()/]  
[collection->isEmpty()/]  
[collection->notEmpty()/]  
[collection->first()/]  
[collection->last()/]  
[collection->select(x | condition)/]  
[collection->reject(x | condition)/]  
[collection->collect(x | expression)/]  
[collection->forAll(x | condition)/]  
[collection->exists(x | condition)/]
```

Strings:

```
[string.toUpperCase()/]  
[string.toLowerCase()/]  
[string.toUpperCaseFirst()/]  
[string.substring(start, end)/]  
[string.concat('suffix')/]  
[string.size()/]
```

Conditionals:

```
[if (condition)]...[elseif (condition)]...[else]...[/if]  
[if (x = y)]...[/if]  
[if (x <> y)]...[/if]  
[if (x > y)]...[/if]
```

File Generation Patterns**Simple File:**

```
[file ('output.txt', overwrite)]  
  Content  
[/file]
```

Dynamic Filename:

```
[file (aClass.name + '.java', overwrite)]  
  public class [aClass.name/] { }  
[/file]
```

Subdirectory:

```
[file ('com/example/' + aClass.name + '.java', overwrite)]
package com.example;
public class [aClass.name/] { }
[/file]
```

Conditional Generation:

```
[for (cls : Class | model.eAllContents(Class))]
  [if (not cls.isAbstract)]
    [file (cls.name + '.java', overwrite)]
      // Only concrete classes
    [/file]
  [/if]
[/for]
```

Keyboard Shortcuts (Eclipse)

Shortcut	Action
Ctrl+Space	Content assist in .mtl files
Ctrl+Shift+O	Organize imports (Java)
Ctrl+Shift+F	Format code (Java)
F3	Go to definition
Ctrl+/ <td>Toggle comment</td>	Toggle comment
F5	Step into (debugging)
F6	Step over (debugging)
F8	Resume (debugging)
Ctrl+F11	Run last launch
F11	Debug last launch

End of Developer Guide

For additional help, consult:

- README.md (project overview)
- ECLIPSE_SETUP.md (Eclipse-specific details)
- Official Acceleo documentation (links above)

Happy coding! 🚀